

Beat Tracking as an Order-Constrained Subset Selection Problem

Abstract

We formulate beat and downbeat detection as a discriminative problem in which a fixed number of candidate predictors independently regress candidate event times and classify them, deliberately overgenerating more candidates than there are true events. Training such a model requires a correspondence procedure that decides which candidates are held responsible for which ground-truth events, so that a differentiable classification and regression loss can be computed. We show that once the ground-truth event sequence is indexed by its natural temporal order, and the candidate times are made strictly increasing by an architectural reparameterization, the correspondence procedure reduces to a choice of *which subset of candidates is active* — an order-constrained subset-selection problem, since the pairing between the active candidates and the sorted ground truth is then forced rather than searched over. This subset-selection problem is solved exactly by an $O(NM)$ dynamic program, using a per-pair cost corrected, from the outset, to include the σ -dependent background term the training loss actually requires — not a matched-pairs-only cost patched in afterward. We state the conditions under which the resulting solution is optimal without qualification, and the conditions under which the order constraint is a deliberate modeling choice rather than a free simplification, and we specify the resulting training loss. We further diagnose why downbeat detection is systematically harder than beat detection under this and other formulations, and propose three targeted adjustments — a candidate-level self-attention pass restricted to the classification branch, class-specific loss reweighting, and a differentiable bar-length periodicity regularizer — to mitigate the gap. This formulation is presented strictly within the discriminative paradigm; it is architecturally and probabilistically distinct from the generative bar-pointer Dynamic Bayesian Network framework and the two should not be conflated in method sections addressing either model.

1 Introduction

Classical beat and downbeat trackers, including Dynamic Bayesian Network (DBN) decoders operating over a bar-pointer state space (Böck, Krebs & Widmer, 2016), treat beat tracking as a left-to-right sequential inference problem: a hidden tempo-and-phase state evolves according to a Markov transition law, and the beat sequence is recovered by Viterbi decoding through this state space. This generative formulation enforces rhythmic regularity by construction, since the transition law directly penalizes implausible tempo changes.

An alternative, purely discriminative formulation asks an encoder to produce a fixed number N of candidate events directly, one per position in a coarse partition of the input fragment, each independently emitting a candidate time and a class label. N is set from a physical bound on tempo rather than from the number of true events in any given fragment, so the architecture deliberately overgenerates: most candidates are expected to represent no event at all.

Overgeneration creates a correspondence problem. The N candidates carry no built-in notion of which one, if any, should represent which true event; nothing in the architecture privileges one candidate over another for a given ground-truth event ahead of training. Before a differentiable loss

can be computed, something must decide which candidates are held responsible for the true events, which are held responsible for producing nothing, and what target each responsible candidate is compared against. This decision is itself the value of an unobserved quantity — a correspondence σ , formalized in Section 4, that is never given as data and must instead be inferred: σ is a genuine latent function, not a hyperparameter or an architectural output, and treating it as such is what licenses the training procedure of Section 7. This paper works out that correspondence procedure directly from the structure of the beat-tracking problem: ground-truth beat and downbeat events are strictly ordered in time, and we show that imposing the corresponding order on the candidates reduces the correspondence problem to selecting an active subset of candidates, solvable exactly and efficiently.

2 Problem Setup and Notation

Let $x = (x_1, \dots, x_T)$ denote a sequence of audio frame features (e.g. log-mel spectrogram frames) drawn from a fragment of audio of fixed duration D , which we call the *window duration*. We reserve *window* for the constant D itself — the fixed span of time the model is built to process in one forward pass, from which N is derived below — and *fragment* for a specific instance of that duration positioned within a piece of music, since, as detailed at the end of this section, that position is chosen differently at training and at inference and should not be described by a single deterministic picture. Let the ground-truth annotation for a given fragment consist of M events

$$y_1, \dots, y_M, \quad y_i = (c_i, t_i),$$

where $c_i \in \{\text{DB}, \text{B}\}$ is the class (downbeat or beat) and $t_i \in [0, 1]$ is the normalized event time, with the events indexed in ascending temporal order,

$$t_1 < t_2 < \dots < t_M.$$

This strict ordering on the ground truth is given, not assumed; it follows from the fact that two distinct onsets cannot occur at the same instant and the indexing is simply a relabeling by time.

A fixed integer $N > M$ denotes the number of candidates produced by the model. Rather than choosing N from the encoder’s temporal resolution, we set it directly from a physical bound on tempo: fixing a dataset-level constant $\text{BPM}_{\max}$ chosen to exceed the fastest tempo expected in the training distribution, and writing $D_{\min}$ for the window duration in minutes,

$$N = \text{BPM}_{\max} \times D_{\min}.$$

Following Foscari et al. (2024), we fix $D = 30$ seconds ($D_{\min} = 0.5$) as the window duration throughout; with $\text{BPM}_{\max} = 200$ this gives $N = 100$. This choice requires no tempo estimate for a given input: the same fixed N covers any fragment of duration D regardless of its actual tempo, at the cost of many candidates predicting $\emptyset$ for passages slower than $\text{BPM}_{\max}$ — a cost the background-class down-weighting in Section 7 is designed to absorb. If instead the tempo of each track were known in advance, N could be set tightly to the track’s own BPM, at the cost of under-provisioning during any faster passage within the track; we adopt the dataset-level $\text{BPM}_{\max}$ throughout for this reason.

Fixing the window duration at $D = 30$ seconds does not mean pieces longer than 30 seconds are excluded; a piece of any length is processed as one or more D -second fragments, positioned differently depending on whether training or inference is being described, and the mechanism at each stage is more specific than the published paper text alone conveys. At training, following

Foscarin et al. (2024), a fragment is a randomly positioned crop: one D -second excerpt is sampled from each piece per epoch, padding pieces shorter than D , and $k = \text{round}(\alpha \cdot l/D)$ random crops (with $\alpha = 0.65$) are drawn from a piece of length $l > D$, rather than one fixed crop or a full non-overlapping tiling — a length-proportional random-cropping scheme, not a hard duration cap, and not a deterministic partition of the piece. The underlying architecture is itself length-agnostic (it uses rotary positional embeddings throughout, with no fixed maximum sequence length), so $D = 1500$ frames at training time is a chosen crop length, not an architectural ceiling; validation and prediction, correspondingly, run on full, uncropped pieces rather than a fixed-length input. At inference on a piece longer than D , the published paper text describes tiling into non-overlapping excerpts and concatenating predictions, but the released implementation is more careful: it chunks at $D = 1500$ frames with overlap, discards a 6-frame border from each chunk edge, and stitches the trimmed chunks together — independently confirmed by a third-party reimplement of the released model. This is a real mitigation of the boundary problem, not merely a restatement of the simple tiling the paper text describes: trimming and discarding a border specifically reduces the influence of edge artifacts on the reported output, rather than reporting a chunk’s raw prediction all the way to its own edge.¹

We adopt the training-time random-crop convention as described. For inference on a piece longer than D , the same border-trim-and-stitch idea does transfer, but not by directly porting the frame-level operation: $\hat{\sigma}$ itself is not a per-frame quantity — it is a single discrete correspondence internal to one fragment’s own candidate indexing, with no meaning across fragments — so there is no equivalent of “discard the border frames” at the level of σ . What does transfer is trimming and stitching at the level of decoded detections rather than at the level of $\hat{\sigma}$; Section 9.3 gives this construction in full once the necessary inference machinery has been introduced.

3 Prediction Architecture and Monotonic Reparameterization

We assume, as is standard in current beat-tracking architectures, that the encoder is a transformer: a stack of self-attention layers mapping x to a feature sequence $h = (h_1, \dots, h_T)$ in which every h_t has, in principle, access to the entire input fragment through self-attention, rather than only to a local receptive field as in a convolutional or recurrent encoder. This assumption matters for the downbeat-specific discussion of Section 10: the frame-level features h_t are already globally contextualized, so any residual difficulty in recovering bar-level (as opposed to beat-level) structure downstream is not attributable to a limited receptive field, and must be addressed elsewhere in the architecture.

The encoder output has length T , generally different from N , so some fixed operator is needed to produce the N candidate feature vectors $z_1, \dots, z_N$ from h . We write this simply as

$$z_j(\theta, x) = \text{Downsample}(h)_j, \quad j = 1, \dots, N,$$

¹Concretely, at 50 frames/second a 1500-frame chunk spans 30 seconds and a 6-frame border is 120 ms. To make consecutive chunks’ kept regions meet with no gap and no double-counted frame, chunks must overlap by at least $2 \times 6 = 12$ frames, so consecutive chunks start 1488 frames apart rather than 1500. For example, chunk 1 spans frames 0–1499 and keeps frames 0–1493 after discarding its last 6; chunk 2 spans frames 1488–2987 (overlapping chunk 1 on frames 1488–1499) and keeps frames 1494–2981 after discarding 6 frames from each edge. Chunk 1’s kept region ends exactly where chunk 2’s begins. The reason for discarding rather than averaging at the seam is that the two chunks’ predictions for the same frame are not just slightly different: a frame 5 positions from a chunk’s own edge has almost no leftward context within that chunk, while the same frame computed as part of the neighboring, overlapping chunk may have nearly the full 30 seconds of context behind it — the two versions differ systematically in how well-informed they are, not merely by noise, so keeping the better-contextualized one is preferable to blending them.

for any fixed downsampling operator reducing a length- T sequence to a length- N sequence, for instance a strided convolution or a strided average-pooling layer. Nothing in Sections 4 through 9 depends on which such operator is used; it is an implementation choice orthogonal to the correspondence problem, the dynamic program, and the loss, and we do not assign it any geometric or anchoring role. In particular, no candidate is associated with a fixed reference time, and equation (1) below does not reference the position that produced z_j : z_j is a genuine function of the input x , but it is not required to localize $\hat{t}_j$. Each z_j is passed through two prediction heads: a classification head returning logits over $\{\text{DB}, \text{B}, \emptyset\}$, and a scalar regression head returning a raw value $r_j(\theta, x) \in \mathbb{R}$.

We compute the predicted time sequence by a global monotonic reparameterization of the raw regression outputs $r_1, \dots, r_N$:

$$\hat{t}_j(\theta, x) = \frac{\sum_{k=1}^j \text{softplus}(r_k(\theta, x))}{\sum_{k=1}^N \text{softplus}(r_k(\theta, x))}, \quad j = 1, \dots, N, \quad (1)$$

where $\text{softplus}(r) = \log(1 + e^r) > 0$ for every $r \in \mathbb{R}$. Because each summand in the numerator is strictly positive, the sequence is strictly increasing for every value of θ and every input x :

$$\hat{t}_1(\theta, x) < \hat{t}_2(\theta, x) < \dots < \hat{t}_N(\theta, x), \quad \text{for all } \theta, x,$$

and the normalization by the full sum places every $\hat{t}_j$ in $(0, 1)$. Monotonicity is therefore an architectural invariant rather than a property that must be learned or penalized into the network: the network cannot represent a crossing hypothesis, regardless of what values $r_1, \dots, r_N$ take. Write $\hat{p}_j(\cdot \mid x; \theta)$ for the predicted class distribution of candidate j over $\{\text{DB}, \text{B}, \emptyset\}$.

Where $\hat{t}_j$ ends up is determined entirely by training, not by any property of z_j 's extraction. For a matched candidate ($j = \hat{\sigma}(i)$ for some i), the $L1$ term in the training loss (8) directly pulls $\hat{t}_j$ toward the ground-truth time t_i . For an unmatched candidate ($j \notin \text{im}(\hat{\sigma})$), the loss supervises only its class output toward $\emptyset$ and places no target on $\hat{t}_j$ at all, so its predicted time is unconstrained beyond the ordering in equation (1). Whichever downsampling operator produces z_j therefore supplies, at most, an inductive bias on the raw output r_j through whatever locality that operator has — not a constraint on the output, and not a claim that candidate j corresponds to any particular time. Whether a local downsampling operator helps or hurts relative to one with a more global receptive field is an empirical question that the architecture leaves open, not one it settles by construction.

4 The Correspondence Problem

Because the model always emits exactly N candidates and the ground-truth fragment contains $M < N$ true events, training requires deciding which M of the N candidates are held responsible for the true events, and which class and time target each is compared against. Rather than combine a classification penalty and a time penalty by an unmotivated weighted sum, we derive the per-pair cost as the negative log-likelihood of a stated observation model, so that every term in it has an explicit probabilistic meaning.

Fix a candidate j and a ground-truth event $y_i = (c_i, t_i)$, and suppose y_i were generated by two conditionally independent channels reading off candidate j 's outputs: a categorical channel for the class, and a channel treating the true time t_i as a virtual observation of candidate j 's predicted time $\hat{t}_j(\theta, x)$, corrupted by noise of a stated distribution rather than compared to it by an arbitrary norm.

$$C_i \mid \hat{p}_j \sim \text{Categorical}(\hat{p}_j(\cdot \mid x; \theta)), \quad T_i \mid \hat{t}_j \sim \text{Laplace}(\hat{t}_j(\theta, x), b), \quad C_i \perp T_i \mid \hat{p}_j, \hat{t}_j, \quad (2)$$

with Laplace density $f(t_i | \hat{t}_j) = \frac{1}{2b} \exp(-|t_i - \hat{t}_j(\theta, x)|/b)$ and fixed scale $b > 0$. The conditional independence in (2) is stated as an assumption, not derived from anything prior, and it is what licenses factorizing the joint density of y_i into a product of the two channels below. The choice of Laplace rather than Gaussian for the time channel is not a free modeling choice made to fit a predetermined answer: Laplace is the unique distribution whose negative log-density is, up to an additive constant, the $L1$ penalty already required of this term, so it is the distribution forced by the requirement that the resulting negative log-likelihood reproduce an $L1$, rather than an $L2$, regression cost.

Under (2), the joint density of observing y_i given a match to candidate j is

$$p(y_i | \hat{y}_j) = \hat{p}_j(c_i | x; \theta) \cdot \frac{1}{2b} \exp\left(-\frac{|t_i - \hat{t}_j(\theta, x)|}{b}\right),$$

and its negative log is

$$-\log p(y_i | \hat{y}_j) = -\log \hat{p}_j(c_i | x; \theta) + \frac{1}{b} |t_i - \hat{t}_j(\theta, x)| + \log(2b).$$

The additive constant $\log(2b)$ does not depend on θ while b is held fixed, so it may be dropped without affecting the location of any minimizer over θ . Identifying $\lambda_{L1} := 1/b$, we take the per-pair cost to be exactly this negative log-likelihood:

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_j) = -\log \hat{p}_j(c_i | x; \theta) + \lambda_{L1} |t_i - \hat{t}_j(\theta, x)|, \quad (3)$$

which is now, term for term, $-\log p(y_i | \hat{y}_j)$ under (2), rather than an engineered sum. In particular λ_{L1} is not a free balancing weight chosen for convenience; it is the precision (inverse scale) of the assumed time-observation noise, and could in principle be estimated from data — e.g. by a closed-form update setting b to the mean absolute matched residual, in the same spirit as the learnable transition-bandwidth parameters in a bar-pointer DBN — rather than fixed as a hyperparameter, though we do not pursue a learnable b here. This derivation also fixes a discrepancy in the raw-probability convention inherited from the object-detection matching-cost literature: only $-\log \hat{p}_j(c_i | x; \theta)$, not $-\hat{p}_j(c_i | x; \theta)$, is licensed by the observation model in (2), so we use $-\log \hat{p}_j$ throughout, making the per-pair cost and the training loss of Section 7 the same quantity in form, not merely in spirit.

The ground-truth events are sorted by construction (Section 2), and the candidate times are sorted by construction (Section 3, equation (1)). Given two sorted sequences, the only remaining freedom in a correspondence between them is which elements of the longer sequence participate: once a set of M candidates is chosen, pairing the i -th ground-truth event with the i -th smallest chosen candidate is the only pairing consistent with both orderings. The correspondence problem is therefore not a search over pairings at all; it is a search over which candidates to select — and that search is a search over the value of a single unobserved object, σ , defined next.

Definition 1. An order-preserving injection is a strictly increasing function $\sigma : \{1, \dots, M\} \rightarrow \{1, \dots, N\}$, i.e. $\sigma(1) < \sigma(2) < \dots < \sigma(M)$. Write $\mathfrak{D}_{M,N}$ for the set of all such functions. Any $\sigma \in \mathfrak{D}_{M,N}$ pairs the i -th ground-truth event with candidate $\sigma(i)$: $y_i \leftrightarrow \hat{y}_{\sigma(i)}$, for $i = 1, \dots, M$.

σ is a latent function in the ordinary statistical sense of that term: it is never observed, not part of θ , not an architectural output, and not given by the annotation, which supplies only $y_1, \dots, y_M$ themselves, not which candidate stands for which. Everything from this point through Section 7 is the construction of a well-defined estimation problem for this one unobserved function, given the observed $y_{1:M}$ and the candidate predictions $\hat{y}_1, \dots, \hat{y}_N$ that θ and x already determine.

Although σ is a function, $\mathfrak{D}_{M,N}$ is not the space of all injections or permutations: because σ must be strictly increasing, it is completely determined by its image $\{\sigma(1), \dots, \sigma(M)\} \subset \{1, \dots, N\}$, so $|\mathfrak{D}_{M,N}| = \binom{N}{M}$ — the same search space size as choosing which M candidates participate, not $M!$ or any larger permutation-sized count.

The estimation problem below is a minimization over σ alone, not over $\hat{y}_1, \dots, \hat{y}_N$: these N candidate predictions are given at the point $\hat{\sigma}$ is computed, being the network’s own outputs $\hat{y}_j = (\hat{p}_j(\cdot \mid x; \theta), \hat{t}_j(\theta, x))$, already fixed functions of θ and x once the forward pass of Section 3 has run, exactly as $y_1, \dots, y_M$ are given from the annotation. Nothing about $\hat{y}_j$ is chosen or searched over here; only which M of the N already-computed candidates participate, and in what order-preserving pairing with the ground truth, is what σ ranges over.

One further correction belongs here, before the estimation problem is stated, rather than later where it might seem like an afterthought. The training loss actually used to update θ (Section 7) is not $\sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ alone: it additionally penalizes every candidate *not* selected by σ for failing to predict background, at a fixed down-weight $\gamma \in (0, 1]$ motivated in Section 10.3 to counteract the class imbalance created by the $N - M$ inactive candidates. Writing $g_j(\theta, x) := -\log \hat{p}_j(\emptyset \mid x; \theta)$ for the cost of candidate j predicting background, this additional term is $\gamma \sum_{j \notin \text{im}(\sigma)} g_j(\theta, x)$, and because $|\text{im}(\sigma)| = M$ is fixed for every $\sigma \in \mathfrak{D}_{M,N}$, it depends on σ through $\sum_{j \notin \text{im}(\sigma)} g_j(\theta, x) = \sum_{j=1}^N g_j(\theta, x) - \sum_{i=1}^M g_{\sigma(i)}(\theta, x)$ — a genuine, σ -dependent quantity, not a constant that can be dropped from the estimation problem below without changing which σ it selects. If the estimation problem used $\mathcal{L}_{\text{match}}$ alone, its solution would not, in general, be the σ that minimizes the loss actually used to train θ ; there is no compensating benefit to leaving this out, since folding it in changes nothing about the estimation problem’s difficulty. We therefore define the corrected per-pair cost

$$\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) := \mathcal{L}_{\text{match}}(y_i, \hat{y}_j) - \gamma g_j(\theta, x), \quad (4)$$

and use it, rather than $\mathcal{L}_{\text{match}}$ alone, everywhere a correspondence is compared against the training loss — including the estimation problem itself, immediately below, and the dynamic program of Section 5 that solves it.

The estimation problem is

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{D}_{M,N}} \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}). \quad (5)$$

4.1 Learning the Precision Parameter

Fixing b (equivalently $\lambda_{L1} = 1/b$) as a hyperparameter discards information the training data actually carries: b is the scale of the assumed time-observation noise in (2), and its maximum-likelihood value is computable in closed form from the residuals the model itself produces. Recall from Section 2 that x , M , and N are all specific to one fragment of fixed duration D ; everything below is computed for that single fragment, not pooled across the dataset except where a minibatch average is stated explicitly. Given the matched pairs $(t_i, \hat{t}_{\hat{\sigma}(i)}(\theta, x))$ for $i = 1, \dots, M$, with $\hat{\sigma}$ held fixed as in Section 7, the time-channel contribution to the joint negative log-likelihood of this fragment is

$$M \log(2b) + \frac{1}{b} \sum_{i=1}^M |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|.$$

Differentiating with respect to b and setting the result to zero gives

$$\hat{b} = \frac{1}{M} \sum_{i=1}^M |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|, \quad (6)$$

the sample mean absolute residual over matched pairs — the classical maximum-likelihood scale estimate for a Laplace distribution. In practice (6) is computed once per minibatch, or as an exponential moving average across minibatches for stability, and held fixed while θ is updated by gradient descent using the resulting b in (3) and (8). This is an ordinary closed-form maximum-likelihood point estimate of a single scalar parameter, not an expectation over a latent variable’s posterior, and there is no distribution over b being marginalized; it should not be described by EM terminology. What genuinely carries over from Section 7’s treatment of $\hat{\sigma}$ is only the alternation pattern — one block of quantities held fixed while another is optimized to an exact optimum, then swapped — which is more accurately described as block coordinate ascent on (θ, b) .

4.1.1 Why a shared, global b does not admit cheating

Under (6), b is a single scalar shared by every candidate in every fragment of the minibatch, and it is updated only from the aggregate residual, not from any individual pair. No single candidate’s prediction can move $\hat{b}$ enough to materially reduce its own penalty: within any block of gradient steps on θ , b is held fixed, so a poorly localized candidate still incurs $|t_i - \hat{t}_{\hat{\sigma}(i)}|/b$ at whatever b currently is, and the only way to reduce that term is to reduce the residual itself. There is no route by which the network can unilaterally inflate or shrink b to excuse an individual bad prediction; b moves only in response to a genuine, aggregate change in localization accuracy across many matched pairs.

4.1.2 Where cheating becomes possible: per-candidate precision

The situation changes if precision is instead predicted per candidate, $b_j = b_j(\theta, x)$, from a small head reading z_j — a natural extension if one wants the model to express reduced confidence in acoustically ambiguous passages rather than a single dataset-wide noise level. This reopens exactly the degree of freedom the global version closes, and it does so in two directions, both traceable to the same term in the loss.

Holding a single candidate’s residual $|t_i - \hat{t}_j|$ fixed, the per-example time-channel loss is $\ell(b_j) = |t_i - \hat{t}_j|/b_j + \log(2b_j)$, with $\partial\ell/\partial b_j = -|t_i - \hat{t}_j|/b_j^2 + 1/b_j$. Two failure modes follow directly:

Precision inflation on hard candidates. Whenever the current residual exceeds the current precision scale, $|t_i - \hat{t}_j| > b_j$, the gradient $\partial\ell/\partial b_j$ is negative: increasing b_j reduces the loss. If this is an easier direction for gradient descent to move in than reducing the residual itself — plausible early in training, or for a genuinely hard example — the network is rewarded for declaring low confidence rather than for improving $\hat{t}_j$. This is the cheating the precision parameter is exposed to: the model pays a much smaller, easily-purchased penalty by inflating b_j instead of the harder work of localizing correctly. It is self-reinforcing, since the gradient of the residual term with respect to $\hat{t}_j$ itself scales as $1/b_j$; once b_j has inflated, the pressure to correct $\hat{t}_j$ on that candidate weakens further, entrenching poor localization exactly where it is least acceptable.

Precision collapse on easy candidates. In the opposite regime, as $\hat{t}_j \rightarrow t_i$ and the residual shrinks toward zero, $\ell(b_j) \rightarrow \log(2b_j)$, which is unbounded below as $b_j \rightarrow 0$. A candidate that has already learned to localize well can keep reducing its loss indefinitely by driving b_j toward zero, independent of any further gain in accuracy — a numerically unstable, degenerate direction that

can dominate the loss landscape and starve gradient signal from harder candidates elsewhere in the batch.

4.1.3 Mitigations

Three changes address these failure modes without abandoning per-candidate precision where it is wanted. First, bound b_j away from zero and infinity by construction rather than by penalty, reusing the device already used for monotonicity in Section 3:

$$b_j(\theta, x) = b_{\min} + \text{softplus}(u_j(\theta, x)),$$

for a fixed floor $b_{\min} > 0$ and a raw head output u_j ; this rules out collapse by construction, the same way (1) rules out crossing by construction, rather than by hoping a penalty term suppresses it in practice. Second, regularize the remaining freedom with a prior: placing a Gamma prior on the precision $1/b_j$ — the natural conjugate prior for a Laplace scale — and optimizing the resulting maximum a posteriori objective adds a term pulling b_j toward a data-informed default, damping runaway inflation on hard candidates rather than only bounding it. Third, decouple the two gradient roles b_j plays: use a stop-gradient on b_j when computing $\partial\mathcal{L}/\partial\hat{t}_j$, so that a single step cannot simultaneously widen b_j and leave $\hat{t}_j$ unimproved; b_j is then updated only through its own, separate loss term, breaking the joint-optimization feedback loop that makes inflation attractive in the first place. A short warm-up phase training with a fixed global b (as in equation (6) above) before unfreezing any per-candidate head is a cheap complement to all three, since it forces accurate localization to be learned before the network has an uncertainty channel available to substitute for it.

A risk specific to this architecture, beyond the general heteroscedastic-regression pathology above, is that $\mathcal{L}'_{\text{match}}$ is not only a loss term but also the cost driving the subset-selection problem (5) itself, computed before training loss and gradients are ever formed. If b_j is allowed to inflate, the dynamic program of Section 5 minimizes an already-distorted cost, and can select a worse-fitting $\hat{\sigma}$ precisely because its inflated-precision-adjusted apparent cost looks artificially cheap — corrupting the correspondence, not merely the loss value computed after it. For this reason, if per-candidate precision is used at all, we recommend computing $\mathcal{L}'_{\text{match}}$ for the purpose of solving (5) using the shared, periodically updated b from (6) even when the training loss (8) afterward uses the per-candidate b_j — insulating the discrete selection step from a pathology that, by the argument above, is confined to the continuous regression head.

5 Solving the Subset-Selection Problem by Dynamic Programming

Equation (5) is an instance of constrained subsequence selection, structurally the same recursion used for weighted longest-increasing-subsequence problems. Computing (5) by directly enumerating all $\binom{N}{M}$ correspondences is intractable for realistic N and M . Because σ must be order-preserving, however, the problem breaks into smaller copies of itself: which candidate is matched to ground-truth event i , and how events $1, \dots, i-1$ are matched, can be decided without knowing how events $i+1, \dots, M$ will be matched, given only how many candidates remain available afterward. This makes an incremental solution possible, one candidate at a time, provided we track not only the answer to the full problem but the answer to every smaller version of it along the way — the minimum cost of matching just the first i ground-truth events using only the first j candidates, for

every i and j . Define, for $0 \leq i \leq M$ and $0 \leq j \leq N$, the table

$D[i, j]$ = minimum cost of an order-preserving injection σ from $\{1, \dots, i\}$ into $\{1, \dots, j\}$.

The boundary conditions are $D[0, j] = 0$ for all j (the empty injection costs nothing), and $D[i, 0] = +\infty$ for $i \geq 1$ (no candidates available cannot cover a nonempty domain). The recursion follows by conditioning on whether j lies in the image of σ :

$$D[i, j] = \min \left(\underbrace{D[i, j-1]}_{j \notin \text{im}(\sigma)}, \underbrace{D[i-1, j-1] + \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)}_{\sigma(i)=j} \right), \quad i \geq 1, j \geq 1. \quad (7)$$

$D[i, j]$ is the answer to a smaller sub-problem for every (i, j) — matching only the first i ground-truth events, using only the first j candidates — and the recursion above fills in every one of these sub-problems, from small i, j up to large. $D[M, N]$ is the one specific cell where $i = M$ (all M ground-truth events) and $j = N$ (all N candidates available) — exactly the original problem, equation (5), with no restriction to a sub-range of events or candidates. The optimal cost is therefore $D[M, N]$: the single number in the table's last cell is, by Proposition 1 below, the true minimum of equation (5) over every $\sigma \in \mathfrak{D}_{M, N}$, not an approximation or a bound on it. $\hat{\sigma}$ is recovered by the standard backtracking argument through the table. Filling the table requires $O(NM)$ time and $O(NM)$ space (reducible to $O(N)$ space with a rolling array if only the cost, not the argmin, is needed).

Equation (7)'s two branches are exactly the two ways of taking a single step through the (j, i) grid below: a rightward step from $(j-1, i)$ to (j, i) when candidate j is skipped, or a diagonal step from $(j-1, i-1)$ to (j, i) when candidate j is matched to event i — never straight up, since i can only advance alongside j . A path built from these two step types, starting at $(0, 0)$ and ending at (N, M) , is what $\sigma \in \mathfrak{D}_{M, N}$ looks like drawn as a staircase; Figure 1 shows the one such path corresponding to a concrete $\hat{\sigma} = (1, 3, 5)$ for $M=3$, $N=5$ — candidates 1, 3, and 5 matched to events 1, 2, and 3, candidates 2 and 4 skipped. $D[i, j]$ is the minimum cost, summed over $\mathcal{L}'_{\text{match}}$ along diagonal steps, of any staircase reaching (j, i) ; $D[M, N]$ is the minimum over every staircase reaching the far corner, and $\text{choice}[i, j]$, recorded during the forward pass, marks which of the two step types the winning staircase used to arrive there.

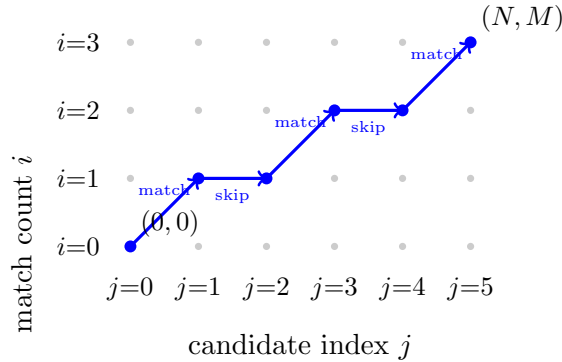


Figure 1: The staircase corresponding to $\hat{\sigma} = (1, 3, 5)$ for $M = 3$, $N = 5$: diagonal steps mark matches, horizontal steps mark skips, and the path runs from $(0, 0)$ to (N, M) by construction.

Algorithm 1 SUBSETSELECTDP — order-constrained subset selection via dynamic programming

Require: ground truth $y_1, \dots, y_M$; candidates $\hat{y}_1, \dots, \hat{y}_N$; corrected cost function $\mathcal{L}'_{\text{match}}$, equation (4)

```
1: Initialize  $D[0, j] \leftarrow 0$  for  $j = 0, \dots, N$ ;  $D[i, 0] \leftarrow +\infty$  for  $i = 1, \dots, M$ .
2: for  $i = 1, \dots, M$  do
3:   for  $j = 1, \dots, N$  do
4:      $c_{\text{skip}} \leftarrow D[i, j-1]$ ;  $c_{\text{match}} \leftarrow D[i-1, j-1] + \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)$ 
5:     if  $c_{\text{match}} < c_{\text{skip}}$  then
6:        $D[i, j] \leftarrow c_{\text{match}}$ ;  $\text{choice}[i, j] \leftarrow \text{MATCH}$   $\triangleright$  records  $\sigma(i) = j$ 
7:     else
8:        $D[i, j] \leftarrow c_{\text{skip}}$ ;  $\text{choice}[i, j] \leftarrow \text{SKIP}$   $\triangleright$  records  $j \notin \text{im}(\sigma)$ 
9:     end if
10:   end for
11: end for
12:  $i \leftarrow M$ ;  $j \leftarrow N$   $\triangleright$  backtracking pointers, reusing the names above only after their loops have exited
13: while  $i > 0$  do
14:   if  $\text{choice}[i, j] = \text{MATCH}$  then
15:      $\hat{\sigma}(i) \leftarrow j$ ;  $i \leftarrow i - 1$ ;  $j \leftarrow j - 1$ 
16:   else  $\triangleright \text{choice}[i, j] = \text{SKIP}$ 
17:      $j \leftarrow j - 1$ 
18:   end if
19: end while
20: return  $\hat{\sigma}$ 
```

Candidates not in the image of $\hat{\sigma}$ are, by construction, treated as predicting the background class $\emptyset$.

6 Optimality and Scope of the Order Constraint

Proposition 1 (Exactness of the dynamic program). *The recursion (7) computes $D[M, N]$ equal to the true minimum in (5) over all order-preserving injections $\sigma \in \mathfrak{D}_{M, N}$.*

Proof sketch. By induction on i . The claim holds trivially for $i = 0$. Assume $D[i-1, j']$ is the true minimum cost of an order-preserving injection from $\{1, \dots, i-1\}$ into $\{1, \dots, j'\}$, for every $j' < j$. Any order-preserving injection from $\{1, \dots, i\}$ into $\{1, \dots, j\}$ either has $j \notin \text{im}(\sigma)$, in which case its cost is bounded below by $D[i, j-1]$, or has $\sigma(i) = j$, in which case its restriction to $\{1, \dots, i-1\}$ is an order-preserving injection into $\{1, \dots, j-1\}$, whose cost is bounded below by $D[i-1, j-1]$ by the inductive hypothesis, plus the cost $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)$ of setting $\sigma(i) = j$. The recursion takes the minimum of these two cases, so $D[i, j]$ is the true minimum over injections into $\{1, \dots, j\}$. $\square$

This establishes that the dynamic program solves (5) exactly, with no approximation, for whatever cost function is used in place of $\mathcal{L}'_{\text{match}}$ — the proof uses only that it is some well-defined per-pair cost, not its specific form. What remains open is a separate question: whether requiring the correspondence to be order-respecting in the first place ever discards a lower-cost correspondence that violates order.

Remark 1. *If the terms depending on the classification head were removed from $\mathcal{L}'_{\text{match}}$ — both $-\log \hat{p}_j(c_i \mid x; \theta)$ and the background term $-\gamma \log \hat{p}_j(\emptyset \mid x; \theta)$ it now carries — leaving only the*

pure time term $|t_i - \hat{t}_j|$, a standard exchange argument for sorted real-valued sequences shows that no order-violating correspondence can achieve a strictly lower total cost than the best order-respecting one: swapping a crossing pair of assignments never increases the total absolute error when both underlying sequences are sorted. Under the full cost (4), this guarantee does not automatically carry over, because an order-violating correspondence could in principle achieve a lower total classification-and-background cost than any order-respecting one, for an untrained or unusually structured network. Equation (5) would then no longer be guaranteed to reach the same value as an unconstrained correspondence search.

We nonetheless adopt the order constraint, for three reasons stated as assumptions rather than proven facts. First, the architectural reparameterization of Section 3 already forbids the network from producing a crossing hypothesis, so an order-respecting correspondence is the only kind consistent with what the network can express in the first place; an unconstrained search would be searching over correspondences the model cannot realize. Second, for a beat-tracking signal, a ground-truth-to-prediction pairing that violates temporal order is physically nonsensical regardless of classification confidence, so such pairings should not be rewarded even where they are formally cheaper. Third, the $O(NM)$ complexity of the dynamic program is preferable to the higher complexity of an unconstrained correspondence search at training time. The second point in particular is a modeling judgment, not a mathematical necessity, and should be stated as such if this formulation is used in downstream work.

7 Training Loss

$\hat{\sigma}$ from equation (5) is the exact minimizer of $\sum_i \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ over $\mathfrak{D}_{M,N}$, and this minimization is a posterior mode, though not of the pure observation model (2) alone. Expanding $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) = \mathcal{L}_{\text{match}}(y_i, \hat{y}_j) - \gamma g_j(\theta, x)$ gives $\exp(-\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)) = p(y_i | \hat{y}_j, \theta) \cdot \hat{p}_j(\emptyset | x; \theta)^{-\gamma}$ — the pure per-pair likelihood of (2), multiplied by a factor depending only on candidate j 's own predicted background probability. Taking the product over $i = 1, \dots, M$ and matching to Bayes' rule identifies exactly what this factor is: $\hat{\sigma} = \arg \max_{\sigma} P(\sigma | \theta, x) P(y_{1:M} | \sigma, \theta, x)$ under the non-uniform prior $P(\sigma | \theta, x) \propto \prod_{i=1}^M \hat{p}_{\sigma(i)}(\emptyset | x; \theta)^{-\gamma}$, not the uniform prior a bare reading of (2) alone would suggest. $\hat{\sigma}$ is therefore the posterior mode of the same reweighted model $\mathcal{L}'_{\text{match}}$ already commits to in equation (4) — not a different or unrelated object, and, because Section 11 builds its full posterior from this identical $\mathcal{L}'_{\text{match}}$, the same model in both cases, not two separate ones. Once obtained, $\hat{\sigma}$ is held fixed — detached from the computational graph, receiving no gradient — and the differentiable training loss for a single fragment is

$$\mathcal{L}(\theta) = \sum_{i=1}^M \left[\omega_{c_i} (-\log \hat{p}_{\hat{\sigma}(i)}(c_i | x; \theta)) + \lambda_{L1} |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)| \right] + \gamma \sum_{j \notin \text{im}(\hat{\sigma})} -\log \hat{p}_j(\emptyset | x; \theta), \quad (8)$$

where $\gamma \in (0, 1]$ down-weights the background-class term to counteract the class imbalance created by the $N - M$ inactive candidates, and $\omega_c > 0$ is a fixed, class-specific weight on the matched classification term, with $\omega_B = 1$ as the reference scale. Setting $\omega_{DB} = 1$ recovers unweighted per-class treatment; Section 10.3 discusses why $\omega_{DB} > 1$ is warranted. The selection cost of equation (5) and the training loss above now share the same background term, γ -weighted and built from the same $g_j(\theta, x)$, by construction of $\mathcal{L}'_{\text{match}}$ in Section 4; the training loss departs from that shared cost only in the class-specific weight ω_{c_i} on the matched term, an explicit importance-reweighting choice to counteract class imbalance (Section 10.3), not a consequence of the observation model in (2).

Because $\hat{\sigma}(\theta, x)$ is recomputed by dynamic programming at every forward pass, this training procedure alternates a non-differentiable selection step, evaluated under the current parameters, with a differentiable parameter update under $\hat{\sigma}$ held fixed. This is, correctly, an instance of hard-EM: the selection step is an E-step that collapses the posterior $P(\sigma \mid y_{1:M}, \theta, x)$ to a point mass at its mode $\hat{\sigma}$ — the same restricted-family construction known elsewhere as Classification EM (Celeux & Govaert, 1992) or Viterbi training, both discussed as instances of one unifying view of EM by Roche (2012), and equivalently describable as an instance of Dirac EM, the posterior collapsed to $\delta_{\hat{\sigma}}$ rather than kept in full — and the parameter update is the corresponding M-step, evaluating the expected complete-data objective under that point mass, which reduces exactly to $\hat{\sigma}$ held fixed while θ is updated. σ ’s status as a genuinely latent function, established in Section 4, is what makes this description correct rather than merely evocative: there is a real posterior being collapsed, not an analogy to one.

Stated in the usual two lines of hard-EM, at iteration k :

$$\hat{\sigma}^{(k)} = \arg \max_{\sigma \in \mathcal{D}_{M,N}} P(\sigma \mid y_{1:M}, \theta^{(k)}, x) = \arg \min_{\sigma \in \mathcal{D}_{M,N}} \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) \quad (\text{E-step}) \quad (9)$$

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_{\theta} \mathcal{L}(\theta) \Big|_{\sigma=\hat{\sigma}^{(k)}} \quad (\text{M-step}) \quad (10)$$

with $\mathcal{L}(\theta)$ as given in equation (8), the posterior $P(\sigma \mid y_{1:M}, \theta, x)$ that of the reweighted model identified above, and $\hat{\sigma}^{(k)}$ held constant when the M-step’s gradient is taken.

7.1 Training with Beat-Only Annotations

Some training data — for instance the SMC dataset used by Foscari et al. (2024) — provides only beat annotations, with no downbeat label. For a matched candidate $j = \hat{\sigma}(i)$ drawn from such data, c_i is known only to lie in the superset $\{\text{B}, \text{DB}\}$, not as a fully observed singleton. This is a partial-label learning problem, distinct from multiple-instance learning: the ambiguity is in which of two candidate labels applies to one instance, not in which of several instances a label applies to.

The marginal likelihood, and what it alone does not supply. The direct construction marginalizes the classification term over the superset rather than committing to either label. Since $\{\text{B}, \text{DB}, \emptyset\}$ exhausts the simplex,

$$\ell_i(\theta) = -\log \left(\hat{p}_j(\text{B} \mid x; \theta) + \hat{p}_j(\text{DB} \mid x; \theta) \right) = -\log \left(1 - \hat{p}_j(\emptyset \mid x; \theta) \right). \quad (11)$$

This term depends on $\hat{y}_j$ only through the sum $\hat{p}_j(\text{B} \mid x; \theta) + \hat{p}_j(\text{DB} \mid x; \theta)$, so its gradient with respect to any perturbation that redistributes probability between B and DB while holding that sum fixed is exactly zero. Equation (11) therefore supplies no training signal whatsoever toward distinguishing downbeat from beat on this specific instance; it only reinforces that the candidate is beat-type rather than background. Whatever the network learns to tell B from DB is, under this loss alone, entirely inherited from datasets where the label is directly observed, transmitted only implicitly through the shared classification head, and never exploited explicitly for the beat-only instances themselves.

Recovering the missing signal via the shared head’s own posterior. Since the same classification head is trained jointly on data where the B/DB distinction is observed, its current estimate of the relative probability of DB versus B already reflects whatever it has learned from

that labeled data, even when evaluated on an instance whose true label is withheld. This can be used directly as a soft pseudo-label. Compute, detached from the computational graph exactly as $\hat{\sigma}$ is detached elsewhere,

$$q_i(\text{DB}) = \frac{\hat{p}_j(\text{DB} \mid x; \theta)}{\hat{p}_j(\text{DB} \mid x; \theta) + \hat{p}_j(\text{B} \mid x; \theta)}, \quad q_i(\text{B}) = 1 - q_i(\text{DB}), \quad j = \hat{\sigma}(i), \quad (12)$$

and replace (11) with a soft cross-entropy against this pseudo-label,

$$\ell_i(\theta) = -q_i(\text{DB}) \log \hat{p}_j(\text{DB} \mid x; \theta) - q_i(\text{B}) \log \hat{p}_j(\text{B} \mid x; \theta). \quad (13)$$

Here q_i genuinely is a posterior over a real latent variable — the withheld true label — computed under the current parameters and held fixed while (13) updates θ ; this is EM-shaped in the same sense as the phase-marginalization construction below, not merely by loose analogy, and follows the same pattern used classically for training classifiers from a mixture of labeled and unlabeled data (Nigam et al., 2000).

This is a latent-variable model, not two separate heuristics. Equations (11), (12), and (13) are not independent devices; they are the marginal likelihood, the E-step posterior, and the M-step surrogate of a single missing-data model built directly on the observation model of Section 4, with no new assumption introduced. Let Z_i denote the same categorical class object as C_i in equation (2), now partially censored: for a beat-only-annotated instance we observe $Z_i \neq \emptyset$ but not which of $\{\text{B}, \text{DB}\}$ it is. Equation (11) is exactly the log-likelihood of this observation, $\log P(Z_i \neq \emptyset \mid x; \theta)$ — not a naive approximation, but the exact marginal likelihood of what is actually known. Equation (12) is exactly the posterior $P(Z_i = \text{DB} \mid Z_i \neq \emptyset, x; \theta)$ under the current parameters, and equation (13) is exactly the negative of the standard EM surrogate $Q(\theta; \theta^{\text{old}}) = \mathbb{E}_{Z_i \sim q_i^{\text{old}}} [\log \hat{p}_j(Z_i \mid x; \theta)]$. By Fisher’s identity, a single gradient step on (13) with q_i frozen at θ^{old} gives exactly the same update as differentiating (11) directly at θ^{old} ; the two constructions diverge only if multiple gradient steps are taken on (13) before q_i is recomputed, the same gap that separates EM from plain gradient descent throughout this document. The zero-gradient property noted above is therefore a statement about the isolated single-instance objective, not a deficiency in (11) itself: the marginal likelihood is exactly correct, and the E-step/M-step decomposition does not add information it lacked, it only makes the cross-instance transfer through the shared parameters explicit and permits multiple surrogate steps per posterior update.

Remark 2 (This is not a latent-variable formulation of subset selection). *The latent variable Z_i introduced above operates strictly downstream of, and conditional on, an already-fixed $\hat{\sigma}$ from Section 5: it concerns only the class label of a pair that the hard dynamic program has already matched, not which pairing was chosen. σ itself remains, throughout this subsection, exactly what Section 5’s Proposition and Section 7 establish it to be — the exact mode of the posterior $P(\sigma \mid y, \theta, x)$ derivable from Section 4’s observation model under a uniform prior, held fixed and detached from the gradient while it serves that role. These are latent variables at two different levels of the same pipeline, answering two different questions: Z_i asks which of two known labels a given matched pair has; σ asks which candidates are matched at all. Conflating the two would misstate what Section 5’s DP computes. Section 11 constructs a model in which the full posterior over σ , not only its mode, is used; it is a different, more invasive construction than the one in this subsection, not a restatement of it.*

8 A Known-Meter Spacing Constraint on σ

The constructions in Section 7.1 (equation (11), introduced above) and Section 10.4 (equation (17), introduced later in this document) use a known meter length L , but neither uses it to influence which σ is selected: Section 7.1’s use is confined to imputing a missing class label after $\hat{\sigma}$ is already fixed, and Section 10.4’s regularizer is added to the training loss only after $\hat{\sigma}$ is fixed as well. Here we incorporate the same known- L information directly into the cost that Section 5’s dynamic program minimizes, so that the spacing constraint can affect the selection itself; we anticipate the local-beat-period estimate $\bar{\Delta}(\theta, x)$ defined precisely in Section 10.4 rather than redefine it here.

Assume full annotation — both class and time are known for every y_i , unlike Section 7.1’s setting — and a known, constant meter L . Let $i_1 < i_2 < \dots < i_K \subset \{1, \dots, M\}$ be the ground-truth indices with $c_{i_k} = \text{DB}$; by definition of a constant meter, $i_{k+1} - i_k = L$ for every consecutive pair. We augment the per-pair cost with an additional term, active only when $i = i_{k+1}$ for some k , penalizing deviation of the predicted downbeat-to-downbeat gap from L times the estimated local beat period $\bar{\Delta}(\theta, x)$ defined in equation (16):

$$\mathcal{L}_{\text{meter}}(\sigma(i_k), \sigma(i_{k+1})) = \mu \left(\hat{t}_{\sigma(i_{k+1})}(\theta, x) - \hat{t}_{\sigma(i_k)}(\theta, x) - L \bar{\Delta}(\theta, x) \right)^2, \quad \mu \geq 0. \quad (14)$$

Unlike every per-pair cost used elsewhere in this document, (14) depends on two matched positions jointly, $\sigma(i_k)$ and $\sigma(i_{k+1})$, rather than on $\sigma(i)$ in isolation. This is a genuine obstacle only because of what Section 5’s dynamic program is: it fills a table $D[i, j]$, defined there as the minimum cost of matching the first i ground-truth events using only candidates $1, \dots, j$, via the recursion of equation (7), in which $D[i, j]$ is computed from $D[i, j-1]$ and $D[i-1, j-1]$ alone. That recursion’s defining property is first-order Markovian: $D[i, j]$ depends on the past only through $D[i-1, j-1]$, with no memory of which specific candidate was matched at any earlier index, only of how many were. Equation (14), needing to know $\sigma(i_k)$ specifically at the later step $i = i_{k+1} = i_k + L$, asks for exactly the memory this recursion does not keep.

An exact but more expensive augmented recursion. The goal is to compute the exact minimizer of the meter-augmented total cost — the original per-pair cost sum of equation (5) plus the meter-spacing penalty (14) summed over every consecutive pair of matched downbeats,

$$\hat{\sigma}_{\text{meter}} = \arg \min_{\sigma \in \mathfrak{D}_{M,N}} \left[\sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) + \sum_{k=1}^{K-1} \mathcal{L}_{\text{meter}}(\sigma(i_k), \sigma(i_{k+1})) \right],$$

over the same feasible set $\mathfrak{D}_{M,N}$ as before — not an approximation to this quantity, and not a different objective. The obstacle is that the basic recursion (7)’s state (i, j) retains only how many events have been matched by candidate j , not which specific candidate was matched to the most recent downbeat — exactly the information $\mathcal{L}_{\text{meter}}$ needs to evaluate a spacing penalty once a later downbeat is reached. The dependency can be restored by carrying the identity of the most recent downbeat’s matched candidate as extra state. Define, for $0 \leq i \leq M$, $0 \leq j \leq N$, and $0 \leq j^* \leq N$,

$D[i, j, j^*]$ = minimum cost of $\hat{\sigma}_{\text{meter}}$ ’s objective, restricted to an order-preserving injection from $\{1, \dots, i\}$ into $\{1, \dots, j\}$

whose most recent downbeat was matched to candidate j^* ,

with the recursion following (7) unchanged at every non-downbeat index, and, at $i = i_{k+1}$, adding $\mathcal{L}_{\text{meter}}(j^*, j)$ to the branch that sets $\sigma(i_{k+1}) = j$, using whichever j^* that branch’s history carries.

This computes the exact minimizer of the augmented cost, but the extra state dimension raises complexity: since j^* ranges over N values but only needs to be tracked between consecutive downbeat indices (there is nothing to remember it against in between), the cost is $O(N^2)$ per inter-downbeat segment rather than $O(N)$, giving $O(N^2M/L)$ overall rather than Section 5’s $O(NM)$ — a real, non-trivial increase, not a constant-factor one, and one that grows as the meter L shrinks.

Where this sits relative to the two existing constructions. Equation (14) is a strictly more invasive use of known meter than either of the two already in the document: Section 10.4’s regularizer accepts whatever $\hat{\sigma}$ the unconstrained DP already produced and only shapes the gradient afterward, at $O(NM)$ total cost; Section 7.1’s phase marginalization uses L to resolve a missing label without touching the selection cost at all. The construction here is the only one of the three where the known meter can change which candidates get selected, at the price of the complexity increase above. Which of the three is worth the cost is an empirical question this document does not resolve; we record all three because they trade off exactness of the meter constraint against where in the pipeline it is applied, not because one is derived from another.

9 Training and Inference Procedures

9.1 Training

A single training step for one fragment x with ground truth $y_1, \dots, y_M$ proceeds as follows: the encoder and pooling heads produce the candidate predictions $\{\hat{t}_j, \hat{p}_j(\cdot \mid x; \theta)\}_{j=1}^N$ of Section 3; each candidate’s background cost $g_j(\theta, x)$ and the corrected pairwise costs $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)$ of (4) are formed; the dynamic program of (7) is solved to obtain $\hat{\sigma}$; and the parameters are updated by descending the gradient of the loss (8) evaluated at $\hat{\sigma}$ held fixed.

Algorithm 2 Training step (single window)

Require: audio window x ; ground truth $y_1, \dots, y_M$ with $t_1 < \dots < t_M$; parameters θ ; learning rate η ; weights $\lambda_{L1}, \gamma, \{\omega_c\}$

E-step (lines 1–17): compute $\hat{\sigma}$, equation (5) — a discrete optimum, no gradients involved

- 1: $h \leftarrow \text{Encoder}(x; \theta)$ $\triangleright$ frame-level features
- 2: **for** $j = 1, \dots, N$ **do**
- 3: $z_j \leftarrow \text{Downsample}(h)_j$
- 4: $r_j, \ell_j \leftarrow \text{Heads}(z_j; \theta)$ $\triangleright r_j \in \mathbb{R}$ raw output; ℓ_j class logits
- 5: $\hat{p}_j(\cdot | x; \theta) \leftarrow \text{softmax}(\ell_j)$
- 6: **end for**
- 7: $Z \leftarrow \sum_{k=1}^N \text{softplus}(r_k)$ $\triangleright$ normalizer, equation (1)
- 8: **for** $j = 1, \dots, N$ **do**
- 9: $\hat{t}_j \leftarrow \frac{1}{Z} \sum_{k=1}^j \text{softplus}(r_k)$ $\triangleright$ equation (1)
- 10: $g_j \leftarrow -\log \hat{p}_j(\emptyset | x; \theta)$
- 11: **end for**
- 12: **for** $i = 1, \dots, M$ **do**
- 13: **for** $j = 1, \dots, N$ **do**
- 14: $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) \leftarrow -\log \hat{p}_j(c_i | x; \theta) + \lambda_{L1}|t_i - \hat{t}_j| - \gamma g_j$ $\triangleright$ equation (4)
- 15: **end for**
- 16: **end for**
- 17: $\hat{\sigma} \leftarrow \text{SUBSETSELECTDP}(\mathcal{L}'_{\text{match}})$ $\triangleright$ Algorithm 1

M-step (lines 18–20): update θ with $\hat{\sigma}$ held fixed, equations (8) and (5)’s aftermath

- 18: $\hat{\sigma} \leftarrow \text{STOPGRADIENT}(\hat{\sigma})$ $\triangleright$ no gradient through the selection step
- 19: $\mathcal{L}(\theta) \leftarrow \sum_{i=1}^M \left[\omega_{c_i} \left(-\log \hat{p}_{\hat{\sigma}(i)}(c_i | x; \theta) \right) + \lambda_{L1}|t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)| \right] + \gamma \sum_{j \notin \text{im}(\hat{\sigma})} -\log \hat{p}_j(\emptyset | x; \theta)$ $\triangleright$ eq. (8)
- 20: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$ $\triangleright$ or an Adam-type update

The stop-gradient in line 18 is the discrete analogue of treating a frozen assignment as a constant with respect to θ : $\hat{\sigma}$ enters the loss as data, and only the class probabilities and regressed times of the *matched* slots contribute gradient, exactly as specified in Section 7. Because $\hat{\sigma}(\theta, x) = \arg \min_{\sigma}$ is an argmin over the finite set $\mathfrak{D}_{M,N}$, it is piecewise-constant in θ wherever the minimizer is unique, so this treatment is exact on that region, not an approximation adopted for convenience.

Algorithm 2 depicts a single step: one window, one loss evaluation, one parameter update — exactly the $K = 1$ case of the outer training loop below, which is what is actually run in practice. Where Section 4.1 refers to a quantity “computed once per minibatch,” a minibatch is the group of K single-window computations Algorithm 2’s own steps 1–19 perform, with their losses averaged and θ updated once per minibatch rather than once per window; Algorithm 2 itself should be read as the per-window computation this outer loop is built from, not as the complete training procedure on its own.

Algorithm 3 Outer training loop

Require: training set of (window, ground truth) pairs; initial parameters θ ; minibatch size K ; number of epochs E ; learning rate η ; weights $\lambda_{L1}, \gamma, \{\omega_c\}$

- 1: **for** epoch = 1, $\dots$, E **do**
- 2: partition the training set into minibatches of K windows each, reshuffled every epoch
- 3: **for** each minibatch $B = \{(x_1, y^{(1)}), \dots, (x_K, y^{(K)})\}$ **do**
- 4: **for** $k = 1, \dots, K$ **do**
- 5: $\mathcal{L}_k(\theta) \leftarrow$ Algorithm 2, steps 1–19, applied to window $(x_k, y^{(k)})$ $\triangleright$ per-window loss;
 $\hat{\sigma}_k$ detached as in line 18
- 6: **end for**
- 7: $\mathcal{L}_B(\theta) \leftarrow \frac{1}{K} \sum_{k=1}^K \mathcal{L}_k(\theta)$ $\triangleright$ minibatch-averaged loss
- 8: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_B(\theta)$ $\triangleright$ Algorithm 2’s own line 20, applied once per minibatch; or an Adam-type update
- 9: update $\hat{b}$ via equation (6), aggregating matched residuals over this minibatch $\triangleright$ Section 4.1
- 10: **end for**
- 11: **end for**
- 12: **return** trained parameters θ

9.2 Inference

At inference time there is no ground truth to match against, so the dynamic program of Section 5 plays no role. Each of the N candidates is decoded independently: its predicted class is taken as the arg max of $\hat{p}_j(\cdot \mid x; \theta)$, and it is reported as a detected event only if that class is not $\emptyset$ and its probability clears a fixed confidence threshold τ .

Algorithm 4 Inference: decoding beat and downbeat locations

Require: audio window x ; trained parameters θ ; confidence threshold $\tau \in (0, 1)$

- 1: $h \leftarrow \text{Encoder}(x; \theta)$
- 2: $B \leftarrow \emptyset$ $\triangleright$ output list of detected events
- 3: **for** $j = 1, \dots, N$ **do**
- 4: $z_j \leftarrow \text{Downsample}(h)_j$
- 5: $r_j, \ell_j \leftarrow \text{Heads}(z_j; \theta)$
- 6: $\hat{p}_j(\cdot \mid x; \theta) \leftarrow \text{softmax}(\ell_j)$
- 7: **end for**
- 8: $Z \leftarrow \sum_{k=1}^N \text{softplus}(r_k)$
- 9: **for** $j = 1, \dots, N$ **do**
- 10: $\hat{t}_j \leftarrow \frac{1}{Z} \sum_{k=1}^j \text{softplus}(r_k)$
- 11: $\hat{c}_j \leftarrow \arg \max_{c \in \{\text{DB}, \text{B}, \emptyset\}} \hat{p}_j(c \mid x; \theta)$
- 12: **if** $\hat{c}_j \neq \emptyset$ **and** $\hat{p}_j(\hat{c}_j \mid x; \theta) \geq \tau$ **then**
- 13: append $(\hat{c}_j, \hat{t}_j)$ to B
- 14: **end if**
- 15: **end for**
- 16: **return** B $\triangleright$ already in ascending time order, since $\hat{t}_1 < \dots < \hat{t}_N$

No non-maximum suppression or post-hoc de-duplication step is needed. This follows from two facts that hold regardless of how $z_1, \dots, z_N$ are obtained from the encoder output: exactly one

classification decision is made per candidate index, and $\hat{t}_1 < \hat{t}_2 < \dots < \hat{t}_N$ holds for every θ and x by equation (1). Together these guarantee that no two reported detections can ever coincide or cross in time. The predicted times are points, not intervals — the relevant property of a point sequence is strict ordering, not disjointness — and the guarantee depends only on this ordering and on there being one classifier per candidate, not on any property of the downsampling operator used to produce z_j .

9.3 Stitching Overlapping Fragments at Inference

Algorithm 4 above decodes one fragment in isolation. For a piece longer than D , the fragments produced by the overlap-and-trim scheme of Section 2 must be decoded separately and combined into one detection list for the whole piece. The obstacle raised in Section 2 was that $\hat{\sigma}$ is a discrete correspondence internal to one fragment’s own candidate indexing, with no meaning across fragments — but that obstacle only applies to $\hat{\sigma}$ itself. It does not apply to the decoded output of Algorithm 4, which is already a list of (class,time) pairs, and a time can always be re-expressed on a shared, absolute axis once a fragment’s start offset is known. Stitching should therefore happen at the level of decoded detections, exactly where Foscarin et al. (2024) stitch theirs, not at the level of $\hat{\sigma}$.

Fix a border duration $\beta \in (0, D/2)$, chosen analogously to the 120 ms border of Section 2’s footnote; we return to how β might be set below. Tile the piece into fragments starting at offsets $o_1 = 0, o_2 = D - 2\beta, o_3 = 2(D - 2\beta), \dots$, so consecutive fragments overlap by 2β , mirroring the frame-level scheme exactly but measured in continuous time rather than frame count. Each fragment’s keep region is $[o_k + \beta, o_k + D - \beta]$, except the first fragment’s keep region extends down to $o_1 = 0$ and the last fragment’s keep region extends up to the end of the piece — there is no preceding or following fragment to prefer over the piece’s own boundary.

Algorithm 5 Piece-level inference by stitching overlapping fragments

Require: full piece of duration l ; trained parameters θ ; threshold τ ; fragment duration D ; border

β

```

1:  $B_{\text{piece}} \leftarrow \emptyset$ 
2: compute fragment offsets  $o_1, o_2, \dots, o_K$  with stride  $D - 2\beta$  covering  $[0, l]$ 
3: for  $k = 1, \dots, K$  do
4:    $x_k \leftarrow$  the  $D$ -second audio fragment starting at  $o_k$  (padded if it extends past  $l$ )
5:    $B^{(k)} \leftarrow \text{INFERENCE}(x_k, \theta, \tau)$  ▷ Algorithm 4, run independently per fragment
6:   determine this fragment’s keep region  $[\ell_k, u_k]$  (edge exceptions for  $k = 1$  and  $k = K$  as above)
7:   for  $(\hat{c}, \hat{t}) \in B^{(k)}$  do
8:      $\hat{t}_{\text{abs}} \leftarrow o_k + \hat{t} \cdot D$ 
9:     if  $\ell_k \leq \hat{t}_{\text{abs}} \leq u_k$  then
10:      append  $(\hat{c}, \hat{t}_{\text{abs}})$  to  $B_{\text{piece}}$ 
11:    end if
12:  end for
13: end for
14: sort  $B_{\text{piece}}$  by  $\hat{t}_{\text{abs}}$ 
15: return  $B_{\text{piece}}$ 
```

By construction the keep regions of consecutive fragments partition $[0, l]$ with no gap and no overlap, so every absolute time in the piece is the responsibility of exactly one fragment, and a

detection surviving the filter is never duplicated or dropped purely because of where the boundary happened to fall.

What this resolves, and what it does not. This closes the gap flagged in Section 2: there is now a fully specified procedure for combining fragment-level output into one piece-level result, built entirely from quantities already defined in this document (Algorithm 4, the fragment offsets of Section 2, and ordinary time arithmetic), with no new machinery over $\hat{\sigma}$ itself required. It does not eliminate the deeper problem underlying it. A true event lying inside fragment k ’s keep region is only reported if fragment k ’s own $\hat{\sigma}$ happens to detect it; if that fragment’s correspondence problem misses or misclassifies it — for reasons unrelated to stitching, such as the reduced context near that fragment’s own edge — the event is lost even though a neighboring fragment, for which the same event falls in its discarded border region, might have detected it correctly. This is the same residual risk Foscari et al.’s frame-level trimming carries, not a new one introduced here; stitching reduces where boundary errors can occur without removing the possibility that one occurs.

The border β is left as a free parameter, exactly as their 120 ms border was: we do not derive its value here. A reasonable starting point, unvalidated, would tie it to the candidate spacing D/N rather than picking it independent of the architecture, since D/N sets the resolution the correspondence problem already operates at; whether this or some other choice performs best is an empirical question outside the scope of this document.

10 Why Downbeat Detection Is Harder, and Formulation Adjustments

10.1 Diagnosis

Beat detection is fundamentally a periodicity-estimation problem: a beat is any point in a train of roughly evenly spaced onsets, and onset-strength features computed directly from the signal correlate strongly with beat locations at essentially any tempo and in essentially any genre. Downbeat detection asks a different question given the beat grid: which beat is beat one of the bar. This is a question about metrical structure — a coarser periodicity of period L (the bar length) nested inside the beat periodicity — and metrical structure is frequently not marked acoustically at all. A snare backbeat on beat two, a bassline anticipating the downbeat, or a passage with no dynamic accent pattern gives a local feature almost nothing to key on for phase within the bar, even when the beat period itself is estimated correctly. Four factors compound this gap in practice. First, the acoustic correlates of downbeat position — harmonic change, bass emphasis, dynamic accent — are weak and inconsistent across genres, unlike the near-universal correlation between onset strength and beat position. Second, downbeats are roughly $1/L$ as frequent as beats, so a downbeat classifier sees far fewer positive examples than a beat classifier even after conditioning on an event being present at all. Third, correctly identifying phase within the bar in principle requires context from several consecutive beats, not just the local neighborhood around one candidate; a feature extractor tuned for local onset salience is, by construction, examining the wrong timescale for this decision. Fourth, in decoders that estimate a meter length explicitly, a meter misestimate does not produce an occasional isolated error — it shifts every downbeat prediction in the piece systematically, since bar phase is a global, not local, quantity.

It is important not to overstate this diagnosis in one respect: the metrical-perception literature indicates that the downbeat is, for human listeners, the most salient position in the metrical hierarchy, and that even musically untrained listeners spontaneously align their motor response to

it rather than to a metrically weaker position (Ladinig et al., 2009). Downbeats are not, for a human, an inherently subtle or hard-to-perceive event; the four factors above are difficulties for a discriminative model reading local acoustic features, not evidence that downbeats are weakly marked from a listener’s perspective. This sharpens rather than undercuts the first and third factors: if human listeners identify downbeats reliably despite locally weak or inconsistent acoustic correlates, they are evidently doing so using something beyond local onset salience — top-down integration of harmonic rhythm, phrase structure, and learned genre convention over a timescale of several bars, not an acoustically local marker. A discriminative model with access only to pooled per-candidate features, as in the unmodified architecture of Section 3, has no analogous source of long-range structural information, which is precisely the gap the candidate-level self-attention mechanism of Section 10.2 is intended to narrow.

10.2 A candidate-level self-attention pass for the classification branch

Section 3 assumes a transformer encoder, so each frame feature h_t already has, in principle, access to the whole input window through self-attention before downsampling into z_j . This does not by itself resolve the third factor above: self-attention over raw audio frames is under no obligation to organize the information it aggregates along the beat grid, and the classification head for candidate j reads only the single feature vector z_j , which may not expose bar-periodic structure even when the frame-level features that produced it were globally contextualized. What is missing is not global context per se, but context organized at the right granularity — one token per candidate, aligned to the beat grid, rather than one token per audio frame.

We address this by adding a second, shallow self-attention block operating over the sequence of pooled candidate features $z_1, \dots, z_N$, used only by the classification branch:

$$\tilde{z}_j(\theta, x) = \text{SelfAttention}(z_1, \dots, z_N)_j, \quad j = 1, \dots, N, \quad (15)$$

with class logits computed from $\tilde{z}_j$ in place of z_j . The regression head is left reading z_j directly, unmodified from Section 3: the argument for keeping the time prediction local is not that it is architecturally confined to a region — it is not, by Section 3 — but that placing $\hat{t}_j$ near a matched ground-truth time is a comparatively local decision to supervise, needing only enough information to disambiguate a single onset’s precise timing, whereas its class, and downbeat status in particular, is a decision that benefits from beat-grid-scale context that z_j alone does not expose. Equation (15) does not disturb the monotonicity guarantee of Section 3, since $\hat{t}_j$ is computed from z_j , not $\tilde{z}_j$, and the ordering $\hat{t}_1 < \dots < \hat{t}_N$ is unaffected by any transformation applied only to the classification branch. In terms of Algorithm 2 and Algorithm 4, this amounts to inserting $\tilde{z}_j \leftarrow \text{SelfAttention}(z_1, \dots, z_N)_j$ immediately after the per-candidate feature-extraction loop and computing ℓ_j from $\tilde{z}_j$ in place of z_j , while r_j continues to be computed from z_j unchanged.

10.3 Class-specific loss reweighting

The class weight ω_{c_i} introduced in equation (8) directly targets the second factor above. Setting $\omega_{\text{DB}} > \omega_{\text{B}} = 1$ increases the gradient contribution of each downbeat classification error relative to each beat classification error, compensating for the fact that a matched downbeat candidate is encountered roughly $1/L$ as often as a matched beat candidate over a training epoch. This is a direct analogue of the background down-weighting γ already present in the loss, applied one level down, to the imbalance among the active classes rather than between active and inactive candidates.

10.4 A periodicity regularizer for downbeat spacing

The first and fourth factors above call for something the classification loss alone cannot supply: an explicit preference for evenly spaced downbeats, which is the discriminative analogue of the tempo-consistency the generative bar-pointer DBN obtains for free from its transition law. Let $i_1 < \dots < i_K$ index the ground-truth events with $c_{i_k} = \text{DB}$ among $y_1, \dots, y_M$, and let $\hat{\sigma}$ be the matched assignment from Section 7, so that $\hat{t}_{\hat{\sigma}(i_1)}, \dots, \hat{t}_{\hat{\sigma}(i_K)}$ are the predicted times of the matched downbeat candidates. Let

$$\bar{\Delta}(\theta, x) = \frac{1}{M-1} \sum_{i=1}^{M-1} (\hat{t}_{\hat{\sigma}(i+1)}(\theta, x) - \hat{t}_{\hat{\sigma}(i)}(\theta, x)) \quad (16)$$

be the mean spacing among all M matched candidates, beats and downbeats together, serving as a differentiable estimate of the beat period. Define

$$R(\theta, x) = \sum_{k=1}^{K-1} \left(\hat{t}_{\hat{\sigma}(i_{k+1})}(\theta, x) - \hat{t}_{\hat{\sigma}(i_k)}(\theta, x) - L \bar{\Delta}(\theta, x) \right)^2, \quad (17)$$

where L is the meter length (e.g. the time-signature numerator), assumed known at the dataset or track level. This term penalizes deviation of consecutive predicted downbeat spacing from L beat periods, and is added to the training loss as $\mathcal{L}(\theta) + \lambda_R R(\theta, x)$ with a fixed weight $\lambda_R \geq 0$. Because R depends on the entire matched downbeat sequence rather than a single ground-truth/candidate pair, it cannot be expressed as a per-pair term in $\mathcal{L}_{\text{match}}$ and is instead added after the correspondence $\hat{\sigma}$ has already been fixed, alongside the rest of $\mathcal{L}(\theta)$ in Algorithm 2: concretely, $\lambda_R R(\theta, x)$ is added to the loss line once $\hat{\sigma}$ is available, reusing the $\hat{t}_{\hat{\sigma}(i)}$ values already computed there.

Remark 3. *A further extension, discussed here but not formalized, would add a small global head estimating meter length $\hat{L}$ and bar phase $\hat{\phi}$ from globally pooled features, analogous to a tempo-estimation head, and use $(\hat{L}, \hat{\phi})$ to bias the per-candidate downbeat logits directly rather than only regularizing predicted spacing after the fact. This is a lower-dimensional, more structured decision than N independent three-way classifications, and it is the point at which this discriminative formulation would most closely resemble the phase state of the generative bar-pointer model — a resemblance that should be flagged explicitly, not treated as incidental, if this extension is pursued, per the paradigm-discipline note in Section 12.*

11 A Model with Genuinely Latent Subset Selection, Trained by Marginal Likelihood

The construction below replaces the hard subset-selection step of Sections 4–7 with a construction that retains the full posterior distribution over σ , rather than only its mode, used to define the training loss itself — not a device applied only after training to interpret or post-process a fixed $\hat{\theta}$. Training under this section’s loss means differentiating a marginal likelihood that sums over every order-preserving injection, rather than differentiating a loss evaluated at one hard-selected σ with the selection step detached from the gradient. Everything about the feasible set is unchanged from Section 4: σ still ranges only over $\mathfrak{D}_{M,N}$, and the monotonicity guarantee of Section 3 is untouched, since nothing here modifies how $\hat{t}_j$ is computed.

Reusing the correction already established. Section 4 defines $g_j(\theta, x) := -\log \hat{p}_j(\emptyset \mid x; \theta)$ and the corrected cost $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) := \mathcal{L}_{\text{match}}(y_i, \hat{y}_j) - \gamma g_j(\theta, x)$ precisely so that the estimation problem (5) — and, by extension, everything built from it below — matches the training loss of Section 7 exactly rather than approximately. We reuse that same $\mathcal{L}'_{\text{match}}$ throughout this subsection, with nothing further to correct.

A distribution over order-preserving injections, and its partition function. Define a Gibbs distribution over $\mathfrak{D}_{M,N}$ using $\mathcal{L}'_{\text{match}}$ as an energy,

$$P(\sigma \mid x; \theta) = \frac{1}{Z(\theta, x)} \exp \left(- \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) \right), \quad Z(\theta, x) = \sum_{\sigma' \in \mathfrak{D}_{M,N}} \exp \left(- \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma'(i)}) \right), \quad (18)$$

the sum in $Z(\theta, x)$ ranging over every one of the $\binom{N}{M}$ order-preserving injections $\sigma' \in \mathfrak{D}_{M,N}$. By the identification already made in Section 7, this is the exact posterior of the reweighted model with prior $P(\sigma \mid \theta, x) \propto \prod_{i=1}^M \hat{p}_{\sigma(i)}(\emptyset \mid x; \theta)^{-\gamma}$ and the pure likelihood of observation model (2) — the same reweighted model $\hat{\sigma}$ is the mode of in Section 7, not a different one. $Z(\theta, x)$ is correspondingly the exact marginal likelihood of that same model, kept in full here rather than collapsed to a point mass.

$Z(\theta, x)$ is computable exactly, in the same asymptotic complexity as Section 5’s dynamic program, by replacing the minimum in the recursion (7) with a log-sum-exp:

$$\tilde{D}[i, j] = \text{logsumexp} \left(\tilde{D}[i, j-1], \tilde{D}[i-1, j-1] - \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) \right), \quad \tilde{D}[0, j] = 0, \quad \tilde{D}[i, 0] = -\infty \quad (i \geq 1), \quad (19)$$

with $\log Z(\theta, x) = \tilde{D}[M, N]$. This is the exact structured-prediction analogue of the relationship between Viterbi decoding and the forward algorithm for hidden Markov models: Section 5’s recursion computes the minimum-cost (most probable) σ under the tropical semiring, while (19) computes the total probability mass over all of $\mathfrak{D}_{M,N}$ under the ordinary sum-product semiring, using the identical recursive structure and the identical $O(NM)$ complexity.

Training loss and procedure. The training loss for a fragment is the negative log marginal likelihood,

$$\mathcal{L}_{\text{marg}}(\theta) = -\log Z(\theta, x) = -\tilde{D}[M, N], \quad (20)$$

dropping the additive, θ -independent constant from the choice of prior over $\mathfrak{D}_{M,N}$. Unlike Section 7’s loss, (20) requires no stop-gradient: every quantity in (19) is differentiable, and $\log Z(\theta, x)$ is backpropagated through directly, exactly as the partition function of a linear-chain conditional random field is backpropagated through its own forward algorithm during training (Lafferty et al., 2001). This is the central sense in which this construction is a training-time procedure: θ is updated by descending the gradient of (20) itself, with the correspondence between predictions and ground truth marginalized out rather than hardened into a detached $\hat{\sigma}$ before any gradient is taken.

Algorithm 6 Training step via marginal likelihood (single window)

Require: audio window x ; ground truth $y_1, \dots, y_M$ with $t_1 < \dots < t_M$; parameters θ ; learning rate η ; weights λ_{L1}, γ

No E-step/M-step split here (contrast Algorithm 2): lines 1–23 build $\tilde{D}[M, N]$, encoding the full posterior $P(\sigma \mid y, \theta, x)$ implicitly rather than freezing it as a discrete object; line 24 differentiates straight through all of it. By Fisher’s identity this gradient equals what a genuine two-phase E-step (freeze the posterior) then M-step (surrogate gradient) would give, but no such phase boundary exists in this computation itself.

- 1: $h \leftarrow \text{Encoder}(x; \theta)$ ▷ frame-level features
- 2: **for** $j = 1, \dots, N$ **do**
- 3: $z_j \leftarrow \text{Downsample}(h)_j$
- 4: $r_j, \ell_j \leftarrow \text{Heads}(z_j; \theta)$
- 5: $\hat{p}_j(\cdot \mid x; \theta) \leftarrow \text{softmax}(\ell_j)$
- 6: **end for**
- 7: $Z_{\text{softplus}} \leftarrow \sum_{k=1}^N \text{softplus}(r_k)$ ▷ normalizer, equation (1)
- 8: **for** $j = 1, \dots, N$ **do**
- 9: $\hat{t}_j \leftarrow \frac{1}{Z_{\text{softplus}}} \sum_{k=1}^j \text{softplus}(r_k)$ ▷ equation (1)
- 10: $g_j \leftarrow -\log \hat{p}_j(\emptyset \mid x; \theta)$
- 11: **end for**
- 12: **for** $i = 1, \dots, M$ **do**
- 13: **for** $j = 1, \dots, N$ **do**
- 14: $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) \leftarrow -\log \hat{p}_j(c_i \mid x; \theta) + \lambda_{L1}|t_i - \hat{t}_j| - \gamma g_j$ ▷ corrected cost, this subsection
- 15: **end for**
- 16: **end for**
- 17: initialize $\tilde{D}[0, j] \leftarrow 0$ for $j = 0, \dots, N$; $\tilde{D}[i, 0] \leftarrow -\infty$ for $i = 1, \dots, M$
- 18: **for** $i = 1, \dots, M$ **do**
- 19: **for** $j = 1, \dots, N$ **do**
- 20: $\tilde{D}[i, j] \leftarrow \text{logsumexp}(\tilde{D}[i, j-1], \tilde{D}[i-1, j-1] - \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j))$ ▷ equation (19)
- 21: **end for**
- 22: **end for**
- 23: $\mathcal{L}_{\text{marg}}(\theta) \leftarrow -\tilde{D}[M, N]$ ▷ equation (20)
- 24: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{marg}}(\theta)$ ▷ backpropagated through every line above; no stop-gradient; or an Adam-type update

This single-window step is invoked by the same outer training loop as Section 7’s construction — many windows per minibatch, many minibatches per epoch — with the sole difference that the per-window computation called inside it is this algorithm rather than Section 7’s, and no analogue of the precision-parameter update of Section 4.1 is shown here since this subsection does not modify how b is estimated.

An equivalent EM perspective exists alongside direct backpropagation, exactly paralleling Section 7.1’s structure: the posterior $P(\sigma \mid y_{1:M}, x; \theta)$ is a genuine distribution obtainable from a forward-backward pass over the same recursion, and by Fisher’s identity a single gradient step on the resulting EM surrogate, with the posterior frozen at θ^{old} , coincides exactly with a single gradient step on (20) directly. The two are optimization strategies for the same objective, not competing models; direct backpropagation through (19) is the simpler of the two to implement and is what we recommend. Sections 7 and 7.2 are consequently two E-steps for one latent-function model, not two separate models of σ : Section 7 collapses $P(\sigma \mid y, \theta, x)$ to a point mass (hard-EM,

equivalently Dirac EM), and this section retains it in full — the underlying latent object is the same σ introduced in Section 4 in both cases.

Constructing the alternative explicitly. The preceding paragraph asserts that a genuine two-phase E-step/M-step version of Algorithm 6 exists; it is worth actually building it, both to confirm the assertion and to make concrete what Algorithm 6’s single differentiable pass is an alternative *to*. This requires one ingredient Algorithm 6 never computes: a backward pass mirroring the forward recursion (19). Define, for $0 \leq i \leq M$, $0 \leq j \leq N$,

$$\tilde{E}[i, j] = \text{logsumexp}(\tilde{E}[i, j+1], \tilde{E}[i+1, j+1] - \mathcal{L}'_{\text{match}}(y_{i+1}, \hat{y}_{j+1})), \quad \tilde{E}[M, j] = 0, \quad \tilde{E}[i, N] = -\infty \ (i < M),$$

filled by decreasing i from $M-1$ to 0 and j from $N-1$ to 0, at the same $O(NM)$ cost as the forward pass. Combining forward, edge, and backward gives the exact posterior probability that ground-truth event i is matched to candidate j :

$$w_{ij} := P(\sigma(i) = j \mid y_{1:M}, \theta, x) = \exp\left(\tilde{D}[i-1, j-1] - \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) + \tilde{E}[i, j] - \tilde{D}[M, N]\right). \quad (21)$$

Freezing w_{ij} and differentiating the resulting weighted sum recovers exactly $\nabla_{\theta} \mathcal{L}_{\text{marg}}(\theta)$, proved directly rather than cited. Write $E_{\sigma}(\theta) := \sum_i \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}; \theta)$, so $Z(\theta, x) = \sum_{\sigma} \exp(-E_{\sigma}(\theta))$ and $\mathcal{L}_{\text{marg}}(\theta) = -\log Z(\theta, x)$. Differentiating Z term by term,

$$\nabla_{\theta} Z(\theta, x) = \sum_{\sigma} \nabla_{\theta} \exp(-E_{\sigma}(\theta)) = - \sum_{\sigma} \exp(-E_{\sigma}(\theta)) \nabla_{\theta} E_{\sigma}(\theta),$$

so that, dividing by $Z(\theta, x)$ and recognizing $\exp(-E_{\sigma}(\theta))/Z(\theta, x) = P(\sigma \mid y, \theta, x)$,

$$\nabla_{\theta} \mathcal{L}_{\text{marg}}(\theta) = - \frac{\nabla_{\theta} Z(\theta, x)}{Z(\theta, x)} = \sum_{\sigma} P(\sigma \mid y, \theta, x) \nabla_{\theta} E_{\sigma}(\theta) = \mathbb{E}_{\sigma \sim P(\sigma \mid y, \theta, x)} [\nabla_{\theta} E_{\sigma}(\theta)],$$

an identity holding for every θ , not only θ^{old} . Expanding $E_{\sigma}(\theta)$ as a sum over i , swapping the two finite sums, and regrouping by the value $\sigma(i) = j$ — every σ sharing that value contributes the identical term $\nabla_{\theta} \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j; \theta)$, so their posterior masses simply add to $w_{ij}(\theta)$ —

$$\nabla_{\theta} \mathcal{L}_{\text{marg}}(\theta) = \sum_{i=1}^M \sum_{j=1}^N w_{ij}(\theta) \nabla_{\theta} \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j; \theta).$$

Evaluated at $\theta = \theta^{\text{old}}$, this is exactly $\nabla_{\theta} Q(\theta)$ at θ^{old} once w_{ij} is frozen at $w_{ij}(\theta^{\text{old}})$: freezing removes the product-rule term that differentiating $w_{ij}(\theta)$ itself would otherwise contribute, and it is exactly this removed term that makes the two constructions coincide. Without the freeze, $\sum_{i,j} w_{ij}(\theta) \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j; \theta) = \mathbb{E}_{\sigma \sim P(\sigma \mid \theta, x)} [E_{\sigma}(\theta)]$, which the standard free-energy identity $\mathcal{L}_{\text{marg}}(\theta) = \mathbb{E}_{\sigma} [E_{\sigma}(\theta)] - H[P(\sigma \mid \theta, x)]$ shows differs from $\mathcal{L}_{\text{marg}}(\theta)$ by the posterior’s own entropy — so the two gradients would differ by $\nabla_{\theta} H[P(\sigma \mid \theta, x)]$, a term generally nonzero, had w not been frozen. STOPGRADIENT(w) in Algorithm 7 is doing exactly this removal, not a stylistic formality.

This necessity is specific to Algorithm 7’s construction, not a general requirement on computing $\nabla_{\theta} \mathcal{L}_{\text{marg}}(\theta)$ — Algorithm 6 is itself a standing counterexample to any broader reading. Algorithm 6 never constructs w as a separate object at all: it differentiates $\mathcal{L}_{\text{marg}}(\theta)$ directly, as one function of θ , and the derivation above shows the correct weighted form $\sum_{i,j} w_{ij}(\theta) \nabla_{\theta} \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j; \theta)$ emerges from ordinary differentiation of $Z(\theta, x)$ with $w_{ij}(\theta)$ tracking the same θ throughout — no freezing decision is available or needed, because w and θ are never separated into two computational stages

in the first place. Freezing matters only once a construction deliberately separates them, computing w at one θ and reusing it while differentiating at another — exactly what an explicit E-step followed by an M-step means, and exactly what Algorithm 6 avoids by not decoupling the two at all.

Algorithm 7 Training step via marginal likelihood, explicit E-step/M-step (single window)

Require: audio window x ; ground truth $y_1, \dots, y_M$ with $t_1 < \dots < t_M$; parameters θ ; learning rate η ; weights λ_{L1}, γ

E-step: compute and freeze the full posterior over σ

- 1: $\{\hat{p}_j, \hat{t}_j, g_j, \mathcal{L}'_{\text{match}}\} \leftarrow$ Algorithm 6, steps 1–16 ▷ candidate predictions and corrected costs
 - 2: $\tilde{D}[i, j] \leftarrow$ Algorithm 6, steps 17–22 ▷ forward pass, equation (19)
 - 3: initialize $\tilde{E}[M, j] \leftarrow 0$ for $j = 0, \dots, N$; $\tilde{E}[i, N] \leftarrow -\infty$ for $i = 0, \dots, M - 1$
 - 4: **for** $i = M - 1, \dots, 0$ **do**
 - 5: **for** $j = N - 1, \dots, 0$ **do**
 - 6: $\tilde{E}[i, j] \leftarrow \text{logsumexp}(\tilde{E}[i, j + 1], \tilde{E}[i + 1, j + 1] - \mathcal{L}'_{\text{match}}(y_{i+1}, \hat{y}_{j+1}))$
 - 7: **end for**
 - 8: **end for**
 - 9: **for** $i = 1, \dots, M$ **do**
 - 10: **for** $j = 1, \dots, N$ **do**
 - 11: $w_{ij} \leftarrow \exp(\tilde{D}[i - 1, j - 1] - \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) + \tilde{E}[i, j] - \tilde{D}[M, N])$ ▷ equation (21)
 - 12: **end for**
 - 13: **end for**
 - 14: $w \leftarrow \text{STOPGRADIENT}(w)$ ▷ freeze the posterior; contrast Algorithm 6, which freezes nothing
 - 15: $Q(\theta) \leftarrow \sum_{i=1}^M \sum_{j=1}^N w_{ij} \mathcal{L}'_{\text{match}}(y_i, \hat{y}_j; \theta)$
 - 16: $\theta \leftarrow \theta - \eta \nabla_{\theta} Q(\theta)$ ▷ provably equals Algorithm 6’s own update, by the identity above
-

Closing synthesis: sumlogexp versus the complete-data log-likelihood, and why EM’s classical motivation does not transfer here. Algorithm 6 differentiates $\mathcal{L}_{\text{marg}}(\theta) = -\log \sum_{\sigma} \exp(-E_{\sigma}(\theta))$ directly — the true, incomplete-data marginal log-likelihood, with σ genuinely summed out. Algorithm 7’s $Q(\theta)$ is exactly $-\mathbb{E}_{\sigma \sim P(\sigma | \theta^{\text{old}})}[\log P(y, \sigma | \theta, x)]$, the complete-data log-likelihood averaged over a frozen posterior. The historical motivation for preferring the latter (Dempster, Laird & Rubin, 1977) is that the former is normally intractable — an exponentially large sum with no efficient closed form — so EM is built specifically to avoid ever evaluating it, working instead with per- σ quantities weighted by a posterior obtained separately. This motivation can be checked rather than assumed, and checking it shows it does not hold for the problem this document poses.

Consider the same estimation problem with σ allowed to range over *all* injections $\{1, \dots, M\} \rightarrow \{1, \dots, N\}$, not only order-preserving ones. Writing $W[i, j] := \exp(-C[i, j])$ for some cost matrix C , the corresponding partition function $Z_{\text{general}} = \sum_{\sigma \text{ injective}} \prod_i W[i, \sigma(i)]$ is, for $M = N$, exactly the permanent of W — $\text{Perm}(W) = \sum_{\sigma \in S_N} \prod_i W_{i, \sigma(i)}$ — and for $M < N$ its natural rectangular generalization. Valiant (1979) proved that computing a matrix permanent exactly is #P-complete, even restricted to 0-1 matrices: a strictly stronger intractability than NP-completeness, with no known polynomial algorithm and none expected barring a complexity-class collapse considered less plausible than $P = NP$. This is precisely the kind of provably-intractable marginal EM’s complete-data trick exists to route around.

The order-preservation constraint changes this completely. At the scale this document’s own Section 2 sets ($N = 100$ from $\text{BPM}_{\text{max}} = 200$, $D = 30\text{s}$; $M \approx 20$ events in a fragment), the

general injection count is $N!/(N - M)! \approx 10^{39.1}$, while the order-preserving count is $\binom{N}{M} \approx 10^{20.7}$ — still astronomically large, shrunk by ”only” eighteen orders of magnitude. And yet the dynamic program of equation (19) computes the *exact* sum over all 5.4×10^{20} order-preserving terms using $NM = 2,000$ arithmetic operations — not an approximation to a hard sum, the exact value of one, computed in a number of steps smaller than the count of terms by twenty orders of magnitude. The difference between a #P-complete marginal and a 2,000-operation one is not a smaller problem; the space is still enormous. It is that order-preservation collapses the general permanent’s unstructured freedom into exactly $O(NM)$ distinct, telescoping sub-problems — the DAG structure of Section 5’s staircase construction — which a permanent has no analogue of.

So: ”the marginal is a mess” is true, rigorously, of the unconstrained bipartite-matching version of this problem; it is false of the problem this document actually solves, and not by luck — the tractability traces directly to the order-preservation the monotonic reparameterization of Section 3 architecturally enforces, three sections before either Algorithm 6 or Algorithm 7 is ever written down. Since the marginal is exactly and cheaply computable, generalized EM’s classical statistical justification — avoid an intractable sum — supplies no reason to prefer Algorithm 7 over Algorithm 6 here. What remains, restated one final time at the correct scope: not better conditioning, not avoided intractability, but the narrower, purely numerical fact that freezing w converts an N -deep backward pass into a flat one, at roughly double the forward cost, for a gradient proved identical either way.

Inference still requires a hard decision, and only one option below is actually available. Marginalizing σ changes what is optimized during training; it does not change the fact that reporting beat and downbeat locations requires a discrete list of times. At inference, decoding reverts to Section 5’s hard dynamic program (Algorithm 1) evaluated at the trained $\hat{\theta}$, exactly as the Inference algorithm of Section 9 already does — the same decoding procedure regardless of which construction, this section’s or Section 7’s, produced $\hat{\theta}$, since decoding depends only on the final θ , not on how training reached it. The alternative this paragraph once offered — thresholding the per-candidate marginal probability of being matched, via the same forward-backward pass used for the EM perspective above — is not actually available at inference: that posterior, $P(\sigma \mid y_{1:M}, x; \theta)$, is conditioned on the ground-truth $y_{1:M}$ used to define it, which does not exist when decoding new, unlabeled audio. It requires correcting, not merely restating: the forward-backward marginal is available only where $y_{1:M}$ happens to be, such as evaluating a trained model against held-out annotations, not as a general inference-time procedure. This mirrors the standard relationship between training and decoding in structured models trained by marginal likelihood — a hidden Markov model trained by Baum–Welch is still decoded by Viterbi at test time, using the observation sequence Viterbi always requires, not a marginal computed from something Viterbi’s own input lacks — and should be read as a statement about how this section’s construction is used, not an oversight: the latent-variable treatment of σ is a training-time device.

Confirmation-bias risk and mitigations. Bootstrapping labels from the model’s own predictions risks reinforcing early mistakes, particularly before the shared head has learned a reliable B/DB distinction from the labeled portion of the training data. Two mitigations, both already used for the same underlying reason elsewhere in this document: a warm-up phase training exclusively on fully-labeled data (or on (11) alone) before introducing (13), mirroring the precision-parameter warm-up of Section 4.1; and confidence gating, using (13) only when $\max(q_i(\text{DB}), q_i(\text{B}))$ exceeds a fixed threshold and falling back to (11) otherwise, so that a genuinely uncertain instance is not force-fit to an unreliable pseudo-label.

A further refinement when meter length is known — and why its premise appears not to hold here. If a piece’s time signature is available even without frame-level downbeat annotation, a stronger construction is possible: for a run of $\hat{M}$ consecutive matched beat-type candidates, exactly one out of every L is a downbeat, with an unknown phase offset $\phi \in \{0, \dots, L-1\}$ — a genuine discrete latent variable with cyclic structure, rather than an independent superset label per instance:

$$P(c \mid x; \theta, \phi) = \prod_{i=1}^{\hat{M}} \hat{p}_{\hat{\sigma}(i)}(c_i(\phi) \mid x; \theta), \quad c_i(\phi) = \begin{cases} \text{DB} & i \equiv \phi \pmod{L} \\ \text{B} & \text{otherwise} \end{cases}, \quad P(c \mid x; \theta) = \frac{1}{L} \sum_{\phi=0}^{L-1} P(c \mid x; \theta, \phi). \quad (22)$$

The corresponding posterior $P(\phi \mid c, x; \theta)$ gives per-instance soft labels mutually consistent across the whole run, rather than independent per-candidate guesses as in (12), at the cost of assuming the matched run contains no missed or spurious candidates, since a single DP correspondence error breaks the phase alignment for every candidate after it. We flag, rather than assume, that this construction’s premise is a real constraint: surveying the eighteen datasets used to train Foscarin et al.’s (2024) system, the two that lack downbeat annotation — Simac and SMC — also lack usable meter metadata, the latter by design, since metrical change and ambiguity are explicitly among the difficulty factors SMC was compiled to contain. The datasets in the same pool that do carry explicit meter metadata, Harmonix and ASAP, already have full downbeat annotation and so never needed this construction. Across this survey, the specific combination equation (22) requires — meter known, downbeat position unknown — does not appear to occur; the construction is retained here as a correct formulation for a hypothetical corpus with that combination (for instance, DJ-tool-tagged audio carrying tempo and time-signature metadata but no frame-level downbeat label), not as a practical option for the specific datasets discussed elsewhere in this document.

12 Discussion and Scope

This formulation is discriminative throughout: $\hat{t}_j$ and $\hat{p}_j$ are deterministic functions of θ and x produced by a feed-forward network, trained by minimizing (8) directly. It contains no hidden Markov state, no transition density over tempo or phase, and no notion of a bar-pointer position. It should not be presented as an alternative parameterization of, nor combined into the method section of, the generative bar-pointer DBN framework; the two make different independence assumptions and answer different questions about the same signal. Where a comparison is warranted, it belongs in a discussion or related-work section, not in a shared derivation.

The monotonicity device of equation (1) is precisely the cumulative-positive-increment construction used to enforce ordered thresholds in cumulative-link ordinal regression models, applied here to a sequence of instance-specific continuous outputs rather than to a shared set of dataset-level cutpoints; the resemblance is at the level of this one reparameterization device only. Ordinal regression predicts a single discrete label against a shared, dataset-level set of ordered cutpoints, whereas the present model regresses N instance-specific continuous times, each independently classified, with the active subset resolved by selection; the two problems should not be conflated beyond the shared reparameterization trick. The subset-selection reduction of Section 4 is structurally related to the monotonic alignment assumption underlying Connectionist Temporal Classification and to Dynamic Time Warping: both replace a search over a large combinatorial space of correspondences with a search constrained by a fixed order, exploitable by the same $O(NM)$ -type dynamic program, for the same complexity-reduction reason.

13 Comparison to a Dense Frame-Classification Baseline

Foscarin, Schlüter, and Widmer’s Beat This! (2024) is a trained, empirically validated discriminative beat and downbeat tracker that, like the present formulation, dispenses with DBN postprocessing; it is instructive to compare the two, not because they solve the same problem the same way, but because the differences trace directly to a single architectural choice. We emphasize at the outset that Beat This! reports state-of-the-art F1 scores across eighteen datasets with full ablations, whereas the present document proposes and analyzes a formulation without a trained model or empirical results; the comparison below is architectural, not a claim of comparative performance.

Beat This! processes 30-second windows through a convolution-and-transformer frontend into a transformer with full self-attention over time, producing two independent per-frame binary logit sequences — beat and downbeat — at the frame rate of the input spectrogram (50 frames per second). Beat locations are recovered by picking local-maximum frames above a probability threshold. Annotation timing imprecision is handled by max-pooling predictions over a small window before computing the loss against frame-synchronous binary labels, so that only the largest nearby prediction is penalized and small shifts go unpunished; class imbalance is handled by weighting the positive-frame term of the binary cross-entropy by the ratio of negative to positive frame counts. Because beat and downbeat are independent output channels, thresholding can produce a downbeat with no coincident beat prediction; a “Sum Head” that adds the downbeat logit into the beat logit before thresholding reduces, but does not eliminate, this inconsistency, and a final post-processing step snaps every downbeat to its nearest beat prediction to guarantee a musically valid output.

The central architectural difference is that Beat This! classifies every frame, while the present formulation regresses a fixed, tempo-bounded number of candidates. This single difference explains most of what follows. Dense per-frame classification needs no correspondence problem: the frame index is the time, so there is nothing analogous to Section 4’s subset selection or Section 5’s dynamic program to solve, and the identity of which output corresponds to which ground-truth event is never ambiguous. The present formulation’s correspondence machinery exists specifically because N candidates without frame-level identity must be matched to M ground-truth events, a problem dense classification sidesteps by construction rather than by solving. A second consequence is where output consistency comes from: Beat This!’s beat and downbeat channels can disagree because they are independently thresholded outputs, requiring the Sum Head and the nearest-beat correction described above as a remedy applied after the fact; in the present formulation a candidate carries a single class label from $\{\text{DB}, \text{B}, \emptyset\}$, so a candidate cannot simultaneously be and not be a beat — consistency is a consequence of the output representation, not a learned or post-processed correction. A third consequence concerns time resolution: Beat This!’s resolution is fixed by the spectrogram hop size regardless of tempo, while the present formulation’s candidate count is set from a tempo bound (Section 2) independent of frame rate, trading a large fixed-resolution output for a smaller, tempo-scaled one with an explicit non-crossing guarantee (equation (1)) that a per-frame classifier has no analogue of.

The two systems address annotation timing imprecision by different means aimed at the same problem: Beat This!’s max-pooled loss is a procedural device with a window size chosen empirically, while the derivation in Section 4 states an explicit observation model and arrives at an interpretable, in-principle-learnable scale parameter (Section 4.1) rather than a fixed pooling width; neither approach has been shown superior to the other, and the comparison should be read as a difference in what each choice makes explicit, not as an argument for one over the other absent empirical evidence. Finally, Beat This! explicitly identifies the same weakness diagnosed independently in Section 10: they report that removing DBN postprocessing degrades continuity metrics specifically because their loss treats each beat individually and does not penalize non-periodic predictions, and

list developing a loss that enforces periodicity during training as future work. This is a point of genuine convergence rather than contrast — an empirical finding on a trained, evaluated system arriving at the same diagnosis reached mechanistically in Section 10.1, and the periodicity regularizer of equation (17) is a candidate answer to precisely the gap they name, though it remains unvalidated here.

References

- [1] S. Böck, F. Krebs, G. Widmer. *Joint Beat and Downbeat Tracking with Recurrent Neural Networks*. ISMIR 2016.
- [2] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber. *Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks*. ICML 2006.
- [3] H. Sakoe, S. Chiba. *Dynamic Programming Algorithm Optimization for Spoken Word Recognition*. IEEE Transactions on Acoustics, Speech, and Signal Processing, 1978.
- [4] O. Ladinig, H. Honing, G. Háden, I. Winkler. *Probing Attentive and Preattentive Emergent Meter in Adult Listeners without Extensive Music Training*. Music Perception, 26(4), 2009.
- [5] F. Foscarin, J. Schlüter, G. Widmer. *Beat This! Accurate Beat Tracking Without DBN Post-processing*. ISMIR 2024.
- [6] K. Nigam, A. McCallum, S. Thrun, T. Mitchell. *Text Classification from Labeled and Unlabeled Documents Using EM*. Machine Learning, 39, 2000.
- [7] J. Lafferty, A. McCallum, F. Pereira. *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICML 2001.
- [8] G. Celeux, G. Govaert. *A Classification EM Algorithm for Clustering and Two Stochastic Versions*. Computational Statistics & Data Analysis, 14(3), 315–332, 1992.
- [9] A. Roche. *EM Algorithm and Variants: An Informal Tutorial*. Spring 2003 (revised September 2012). <https://arxiv.org/pdf/1105.1476>
- [10] L. G. Valiant. *The Complexity of Computing the Permanent*. Theoretical Computer Science, 8(2), 189–201, 1979.